

DBMS PROJECT REPORT

SELLER AND INVENTORY MANAGEMENT SYSTEM

1. Scenario

The e-commerce company sells products supplied by multiple sellers. The system should maintain seller details, track products supplied by each seller, and monitor inventory levels to ensure product availability.

The system uses Seller, Product, and Inventory tables. Seller information is maintained for suppliers, Product information identifies the products being sold, and Inventory information records the available stock for each product.

The main requirements are:

1. Create Seller and Inventory Tables.
2. Establish relationships between Seller, Product, and Inventory.
3. Add Primary Keys to uniquely identify records.
4. Use Foreign Keys to connect related tables.
5. Apply constraints to maintain data integrity.
6. Insert seller, product, and inventory values.
7. Generate seller-wise, stock availability, and inventory analysis reports.

2. Create Seller, Product and Inventory Tables

The database contains tables for Seller, Product, and Inventory. The Seller table stores seller details, the Product table stores product details, and the Inventory table stores stock information.

```
SQL> DROP TABLE Seller CASCADE CONSTRAINTS;
```

Table dropped.

```
SQL> CREATE TABLE Seller (seller_id NUMBER PRIMARY KEY, seller_name VARCHAR2(100) NOT NULL, email VARCHAR2(100) UNIQUE);
```

Table created.

```
SQL> CREATE TABLE Product (product_id NUMBER PRIMARY KEY, product_name VARCHAR2(100) NOT NULL, seller_id NUMBER NOT NULL, CONSTRAINT fk_product_seller FOREIGN KEY (seller_id) REFERENCES Seller(seller_id));
```

Table created.

```
SQL> CREATE TABLE Inventory (inventory_id NUMBER PRIMARY KEY, product_id NUMBER UNIQUE NOT NULL, stock_quantity NUMBER NOT NULL CHECK (stock_quantity >= 0), CONSTRAINT fk_inventory_product FOREIGN KEY (product_id) REFERENCES Product(product_id));
```

Table created.

2.1 Seller Table

The Seller table contains Seller_ID, Seller_Name, and Email. The table structure shown in the SQL output identifies Seller_ID and Seller_Name as required fields.

```
SQL> DESC Seller;
Name                                     Null?    Type
-----
SELLER_ID                               NOT NULL NUMBER
SELLER_NAME                             NOT NULL VARCHAR2(100)
EMAIL                                    VARCHAR2(100)
```

2.2 Product Table

The Product table contains Product_ID, Product_Name, and Seller_ID. Product_ID uniquely identifies each product and Seller_ID connects a product with its seller.

```
SQL> DESC Product;
Name                                     Null?    Type
-----
PRODUCT_ID                               NOT NULL NUMBER
PRODUCT_NAME                             NOT NULL VARCHAR2(100)
SELLER_ID                                 NOT NULL NUMBER
```

2.3 Inventory Table

The Inventory table contains Inventory_ID, Product_ID, and Stock_Quantity. It is used to maintain the available stock for each product.

```
SQL> DESC Inventory;
Name                                     Null?    Type
-----
INVENTORY_ID                             NOT NULL NUMBER
PRODUCT_ID                               NOT NULL NUMBER
STOCK_QUANTITY                           NOT NULL NUMBER
```

3. Establish Relationships

3.1 Seller and Product

One seller can supply multiple products.

The Product table contains Seller_ID to associate each product with a seller.

Relationship: Seller → Product is one-to-many.

3.2 Product and Inventory

Each product has inventory information.

The Inventory table contains Product_ID to associate stock information with a product.

Relationship: Product → Inventory is one-to-one.

4. Primary Key and Foreign Key

Primary keys uniquely identify records in a table. Seller_ID, Product_ID, and Inventory_ID are used to identify records in their respective tables. Foreign keys are used to connect related tables, such as Seller_ID in Product and Product_ID in Inventory.

```
SQL> SELECT p.product_name, i.stock_quantity FROM Product p JOIN Inventory i ON p.product_id = i.product_id WHERE i.stock_quantity = (SELECT MAX(stock_quantity) FROM Inventory);
```

PRODUCT_NAME	STOCK_QUANTITY
Kiwi Fruit	40

5. Insert Values into Seller Table

Seller records are inserted into the Seller table. The records contain seller IDs, seller names, and email addresses.

```
SQL> INSERT INTO Seller VALUES (1, 'Sunshine Grocers', 'sunshinegrocers@gmail.com');
1 row created.

SQL> INSERT INTO Seller VALUES (2, 'Harvest Point', 'harvestpoint@gmail.com');
1 row created.

SQL> INSERT INTO Seller VALUES (3, 'Green Leaf Market', 'greenleafmarket@gmail.com');
1 row created.

SQL> INSERT INTO Seller VALUES (4, 'Nature Basket', 'naturebasket@gmail.com');
1 row created.

SQL> COMMIT;
Commit complete.
```

6. Display Seller Records

The SELECT query is used to display the seller records stored in the database.

```
SQL> SELECT * FROM Seller;
```

SELLER_ID	SELLER_NAME	EMAIL
1	Sunshine Grocers	sunshinegrocers@gmail.com
2	Harvest Point	harvestpoint@gmail.com
3	Green Leaf Market	greenleafmarket@gmail.com
4	Nature Basket	naturebasket@gmail.com

7. Insert Values into Product Table

Product records are inserted into the Product table. Each product record contains a Product_ID, Product_Name, and Seller_ID.

```

SQL> INSERT INTO Product VALUES (101, 'Kiwi Fruit', 1);
1 row created.

SQL> INSERT INTO Product VALUES (102, 'Quinoa', 2);
1 row created.

SQL> INSERT INTO Product VALUES (103, 'Broccoli', 3);
1 row created.

SQL> INSERT INTO Product VALUES (104, 'Red Grapes', 4);
1 row created.

SQL> COMMIT;

Commit complete.

```

8. Display Product Records

The Product table records are displayed using a SELECT query. The output shows product IDs, product names, and the related seller IDs.

```

SQL> SELECT * FROM Product;

```

PRODUCT_ID	PRODUCT_NAME	SELLER_ID
-----	-----	-----
101	Kiwi Fruit	1
102	Quinoa	2
103	Broccoli	3
104	Red Grapes	4

9. Insert Values into Inventory Table

Inventory records are inserted with Inventory_ID, Product_ID, and Stock_Quantity. The stock quantity represents the available inventory for each product.

```

SQL> INSERT INTO Inventory VALUES (1, 101, 40);
1 row created.

SQL> INSERT INTO Inventory VALUES (2, 102, 15);
1 row created.

SQL> INSERT INTO Inventory VALUES (3, 103, 0);
1 row created.

SQL> INSERT INTO Inventory VALUES (4, 104, 30);
1 row created.

SQL> COMMIT;

Commit complete.

```

10. Display Inventory Records

The Inventory table records are displayed using a SELECT query to verify the stored inventory information.

```

SQL> SELECT * FROM Inventory;

INVENTORY_ID  PRODUCT_ID  STOCK_QUANTITY
-----
           1           101             40
           2           102             15
           3           103              0
           4           104             30

```

11. Constraints

Constraints are used to maintain data accuracy and integrity. The database uses required fields, unique email information, primary keys, and foreign key relationships. Stock quantity is also checked so that invalid negative stock values are not accepted.

12. Report 1: Seller-wise Product Report

The Seller-wise Product Report displays seller and product-related information. It can be generated using joins between the related tables.

Display:

- Seller Name
- Number of Products Supplied
- Available Stock

```
SQL> SELECT s.seller_name, COUNT(p.product_id) AS product_count FROM Seller s LEFT JOIN Product p ON s.seller_id = p.seller_id GROUP BY s.seller_id, s.seller_name ORDER BY s.seller_id;
```

SELLER_NAME	PRODUCT_COUNT
Sunshine Grocers	1
Harvest Point	1
Green Leaf Market	1
Nature Basket	1

13. Report 2: Stock Availability Report

The Stock Availability Report displays the current stock quantity and stock status for each product.

Display:

- Product Name
- Stock Quantity
- Stock Status

```
SQL> SELECT s.seller_name, COUNT(p.product_id) AS product_count FROM Seller s LEFT JOIN Product p ON s.seller_id = p.seller_id GROUP BY s.seller_id, s.seller_name ORDER BY s.seller_id;
```

SELLER_NAME	PRODUCT_COUNT
Sunshine Grocers	1
Harvest Point	1
Green Leaf Market	1
Nature Basket	1

14. Report 3: Inventory Analysis

Inventory analysis is performed using SQL queries to obtain information about the total number of products, out-of-stock products, highest stocked products, and average inventory.

14.1 Total Products Available

The total number of products is calculated using COUNT().

```
SQL> SELECT COUNT(*) AS total_products FROM Product;
```

TOTAL_PRODUCTS
4

14.2 Products Out of Stock

Products with a stock quantity of zero are identified as out-of-stock products.

```
SQL> SELECT p.product_name, i.stock_quantity, CASE WHEN i.stock_quantity = 0 THEN 'Out of Stock' WHEN i.stock_quantity <= 10 THEN 'Low Stock' ELSE 'Available' END AS stock_status FROM Product p JOIN Inventory i ON p.product_id = i.product_id ORDER BY p.product_id;
```

PRODUCT_NAME	STOCK_QUANTITY	STOCK_STATUS
Kiwi Fruit	40	Available
Quinoa	15	Available
Broccoli	0	Out of Stock
Red Grapes	30	Available

14.3 Highest Stocked Products

The product or products with the highest stock quantity are identified using an inventory query.

```
SQL> SELECT p.product_name, i.stock_quantity, CASE WHEN i.stock_quantity = 0 THEN 'Out of Stock' WHEN i.stock_quantity <= 10 THEN 'Low Stock' ELSE 'Available' END AS stock_status FROM Product p JOIN Inventory i ON p.product_id = i.product_id ORDER BY i.stock_quantity;
```

PRODUCT_NAME	STOCK_QUANTITY	STOCK_STATUS
Kiwi Fruit	40	Available
Quinoa	15	Available
Broccoli	0	Out of Stock
Red Grapes	30	Available

14.4 Average Inventory Quantity

The average inventory quantity is calculated using the AVG() aggregate function.

```
SQL> SELECT ROUND(AVG(stock_quantity), 2) AS average_inventory FROM Inventory;
```

AVERAGE_INVENTORY
21.25

15. Data Integrity

Data integrity is maintained through primary keys, foreign keys, required fields, and appropriate constraints. These mechanisms help keep seller, product, and inventory records consistent and connected.

16. Conclusion

The Seller and Inventory Management System provides a structured method for maintaining seller information, product information, and inventory levels. The project demonstrates table creation, relationships, keys, constraints, data insertion, record retrieval, and inventory reporting using SQL.

The generated reports provide information about sellers, stock availability, and inventory analysis, helping the database system monitor product stock and maintain organized records.